

Automatización del autocomplete de direcciones

App Passenger (MG-116) · App Driver (MG-117)

Guía de ejecución y reporte de estado · Ambiente **UAT** · Repo `magiis-playwright` (rama `mobile/pax-autocomplete`, MR !66)

Preparado por QA — Emanuel Restrepo · 24 de agosto de 2026

SECCIÓN A · PARA GESTIÓN

SECCIÓN B · PARA QA AUTOMATION

A. Reporte ejecutivo **PARA GESTIÓN**

A.1 · Qué se automatizó

Los dos tickets migran el buscador de direcciones de las apps móviles desde Google Places al endpoint propio de MAGIIS. La automatización verifica que esa migración se comporte según sus criterios: que la app consulte *nuestro* endpoint (y ya no a Google), que agrupe el tecleo, que respete el piso de caracteres, que la sesión de búsqueda se facture como una sola, y que ante errores del servidor degrade sin romper la pantalla.

Suite	App	Casos automatizados y trazados a Xray	Duración típica
<code>test:uat:mg116:all</code>	Passenger	16	10–15 min
<code>test:uat:mg117:all</code>	Driver	10	3–8 min
Total en dispositivo		26	un teléfono, serializado

Cada caso enlaza a su ticket de Xray por anotación `tms`: el resultado de una corrida se consulta en el reporte (Allure / JSON de importación), no preguntándole a quien la corrió.

A.2 · Qué casos están funcionales hoy

Funcionales y en verde: 12 de 26. App Driver: **8 de 10** casos operativos y aprobados en la última corrida. App Passenger: **4 en verde, 1 rojo esperado** (TM-731 reproduce un defecto conocido del producto — que falle es el trabajo del caso), 1 intermitente entre corridas (TM-734) y 10 pendientes. Los pendientes son de *precondición o instrumentación del harness* (pantallas que exigen una navegación que la suite aún no construye), **no defectos del producto**: cada uno lleva su motivo escrito en el artefacto.

Estados de la **última corrida verificada de cada suite** (artefactos `xray-results-mg117-20260824-144841` y `xray-results-mg116-20260824-151921`). Regla de la casa: **un caso sin medición jamás se reporta en verde** — «pendiente» es un resultado honesto, «aprobado sin evidencia» no.

App Driver (MG-117)	Estado	App Passenger (MG-116)	Estado
TM-650 · endpoint propio, cero Google	PASSED	TM-687 · el token rota al seleccionar	PASSED
TM-651 · contrato del request	PASSED	TM-689 · degradación ante 5xx	PASSED
TM-655 · término repetido no re-consulta	PASSED	TM-697 · timeout del endpoint	PASSED
TM-656 · 2 caracteres no consultan	PASSED	TM-729 · Perfil consulta endpoint propio	PASSED
TM-657 · 3 caracteres sí consultan	PASSED	TM-734 · piso de 4 caracteres en Perfil	PASSED (corrida 10:08; en la de 15:19 quedó pendiente)
TM-662 · un sessionToken por sesión	PASSED	TM-731 · recarga en travel-info	FAILED defecto conocido, reproducido
TM-664 · estado vacío controlado	PASSED	10 casos restantes	PENDIENTE
TM-665 · 5xx degrada sin caer a Google	PASSED	Los pendientes de PAX son de instrumentación/precondición (superficies que exigen navegación que la suite aún no construye), no defectos del producto.	
TM-654 · debounce ~300 ms	PENDIENTE medición no atribuible: el intervalo real superó la ventana		
TM-663 · token nuevo tras seleccionar	PENDIENTE exige seleccionar una predicción; gancho aún no provisto		

A.3 · Hallazgo principal: el orden de las sugerencias está mal, y es del backend

Un aeropuerto a miles de kilómetros aparece primero, por delante de la dirección que el usuario tiene al lado. Reproducido en dos capas independientes, con dos apps y dos términos distintos — lo que descarta que sea un problema de una app y lo ubica en el ranking que devuelve el servidor. Lo corrige el ticket de backend **MG-931** (criterio CA-03).

Dónde se midió	Se escribió	Apareció primero	La opción correcta estaba...
App Driver , en el teléfono (2 corridas idénticas, 24-ago)	caza	Aeropuerto «La Macaza» — a 9.168 km	Cazadores 1805, CABA — a 8 km , relegada al 2.º lugar
Capa API (sin teléfono), suite de contrato	corr	Aeropuertos «Corrado Gex» y «Corryong»	Av. Corrientes 4851, CABA — detrás de ambos

La suite de API quedó en rojo *a propósito*: es el criterio de cierre de MG-931. Cuando el fix entre, ese caso pasa a verde sin tocar el test — así el reporte avisa solo.

A.4 · Cómo usar estas automatizaciones

El ciclo completo son cuatro pasos: preparar el teléfono, correr, guardar el artefacto y leer el resultado.

1 · Preparar. Teléfono conectado por USB, Appium corriendo, y **una sola** app MAGIIS abierta — la del actor que vas a probar. Si la otra quedó al frente de una corrida anterior:

```
# para correr la suite del DRIVER:
adb shell am force-stop com.magiis.app.uat.passenger
adb shell am start -W -n com.magiis.app.uat.driver/.MainActivity

# para correr la suite del PASSENGER:
adb shell am force-stop com.magiis.app.uat.driver
adb shell am start -W -n com.magiis.app.uat.passenger/.MainActivity
```

2 · Correr. Desde la raíz del repo `magiis-playwright`:

```
pnpm test:uat:mg117:all      # App Driver      (3-8 min)
pnpm test:uat:mg116:all      # App Passenger (10-15 min)
pnpm test:uat:mg116:api      # contrato del endpoint, sin teléfono (segundos)
```

3 · Guardar el artefacto (si el resultado importa — el reporter pisa la ruta por defecto en cada corrida):

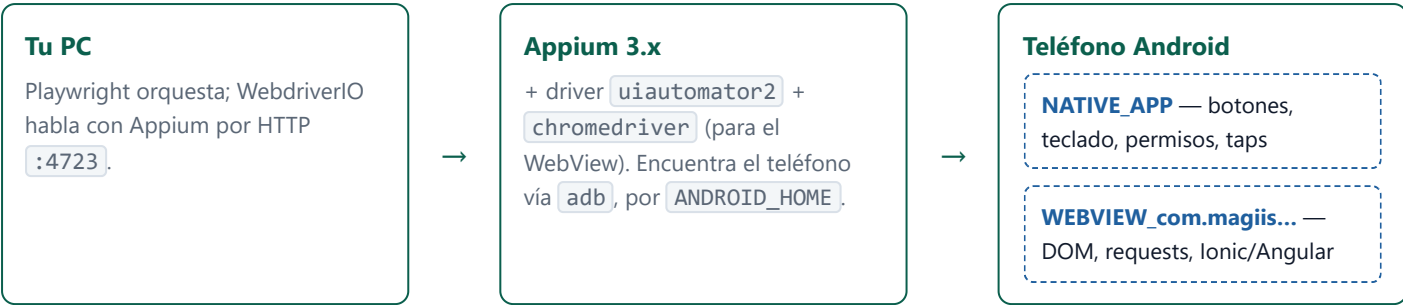
```
$env:XRAY_OUTPUT_FILE = "evidence/uat/xray-results-mg117-$(Get-Date -f yyyyMMdd-HH:mm:ss).json"
pnpm test:uat:mg117:all
```

4 · Leer el resultado. Todo queda en `evidence/uat/`:

Artefacto	Qué es	Cómo se consume
<code>report/</code>	Reporte HTML navegable de la corrida	<code>npx playwright show-report evidence/uat/report</code>
<code>xray-results-*.json</code>	Un estado por caso Xray (PASSED/FAILED/TODO), con el detalle medido en el comentario	Se importa a Xray o se lee directo; es la fuente de la tabla A.2
<code>junit.xml</code>	Resumen por test con el motivo de cada skip	CI / lectura rápida

Requisitos de instalación desde cero: sección B.2. El detalle operativo completo vive en `tests/mobile/appium/README.md` del repo — es la fuente viva; este PDF es la foto del 24-ago.

B.1 · El stack en una vista



Las apps MAGIIS son **híbridas**: lo que se *toca* vive en el lado nativo; lo que se *mide* (requests, DOM) vive dentro del WebView. Cambiar de contexto es explícito, y casi todo diagnóstico difícil termina siendo «estabas en el contexto equivocado».

B.2 · Requisitos y de cero a correr

Requisitos — las versiones con las que esta capa funciona hoy:

Qué	Versión	Para qué
Node.js	20+ (probado en 24.x)	correr el repo — y el render de este PDF
pnpm	11.x	dependencias (<code>pnpm install</code>)
Android SDK + <code>platform-tools</code>	reciente, con <code>ANDROID_HOME</code> exportado	aporta <code>adb</code> ; sin esta variable Appium no ve el teléfono
Java JDK	17+, con <code>JAVA_HOME</code>	lo exige el driver de Android
Appium	3.x (hoy 3.5.2)	servidor de automatización, en <code>:4723</code>
Driver <code>uiautomator2</code>	7.x	<code>appium driver install uiautomator2</code>
Teléfono Android	depuración USB activa y autorizada	el dispositivo de pruebas
Apps MAGIIS del ambiente	<code>com.magiis.app.uat.*</code>	instaladas y con sesión iniciada — los tests no crean cuentas; <code>noReset</code> conserva la sesión
<code>scrcpy</code> (opcional)	cualquiera	ver la pantalla en vivo: la diferencia entre «falló» y «falló porque quedó un modal abierto»

El paso a paso de instalación —con verificación por nivel— está en `tests/mobile/appium/README.md` §1–§5. El checklist mínimo antes de cualquier corrida:

- 1

`adb devices` lista tu equipo en estado `device` — prueba cable y autorización.
- 2

`GET :4723/status` responde — prueba que el *proceso* Appium vive.
- 3

Abrir una sesión real (y cerrarla) — la única verificación de punta a punta. Los dos chequeos anteriores pueden dar verde con un stack que no puede manejar el teléfono (p. ej. un Appium

levantado sin `ANDROID_HOME`).

4

Sólo **una** app MAGIIS abierta: Android depura un único WebView a la vez. Si la suite anterior dejó la otra app al frente, `am force-stop` primero.

B.3 · Anatomía de una corrida

1

Resolución de ambiente: `ENV` decide archivo `.env`, paquete de la app y udid. Cada corrida imprime `[target] OBJETIVO -> env=... package=... udid=...` — léelo siempre.

2

Sesión Appium (~10 s) y cambio al contexto `WEBVIEW_*`.

3

Captura de red inyectada en la página (`webViewNetworkCapture`): engancha `fetch`/XHR para poder afirmar «esta request salió / no salió».

4

Medición por caso: interactuar → esperar → leer la captura → veredicto.

5

Artefactos en `evidence/<env>/`: `junit.xml`, reporte HTML, y el JSON de Xray.

Guardá el artefacto. El reporter escribe siempre la misma ruta y cada corrida pisa la anterior:

```
$env:XRAY_OUTPUT_FILE = "evidence/uat/xray-results-mg117-$(Get-Date -f yyyyMMdd-HHmss).json" antes de correr. Ya perdimos la evidencia de una corrida buena por no hacerlo.
```

B.4 · Disciplina de veredictos — el contrato del equipo

Veredicto	Significa	Qué hacer
PASS	Se midió, y cumple.	Nada.
FAIL	Se midió, y no cumple.	Es un defecto reportable — con el <code>measured</code> del JSON como evidencia.
SIN_DATOS	No hubo nada que medir (p. ej. cero requests).	No es defecto. Buscá por qué no hubo tráfico antes de re-correr.
NO_EJERCIDO	La premisa del caso no se pudo garantizar; un veredicto no sería atribuible.	Leé el motivo escrito: dice exactamente qué premisa falló.

La regla que lo resume: **un verde que no midió nada es peor que un caso pendiente**, porque parece cobertura y no lo es. Los guards de premisa existen para que el harness rehúse opinar antes que inventar un rojo (o un verde).

B.5 · Cómo sumar cobertura a un ticket

1

Investigá primero. Volcá la pantalla real (`pnpm mobile:driver:home-dump` / `mobile:passenger:home-dump`) o usá Appium MCP. No adivines locators.

2

Reusá la pantalla si ya está modelada en `driver/` o `passenger/`.

3

Agregá el caso a la batería del actor, no un archivo nuevo por caso: la batería ya resuelve sesión, navegación y captura de red.

- 4 **Trazalo a Xray** con `annotation: [{ type: 'tms', description: 'TM-xxx' }]` — sin eso el resultado no llega a ningún reporte.
- 5 **Devolvé el estado honesto**: si la premisa no se garantizó, `NO_EJERCIDO` con el motivo escrito.
- 6 **No reuses el término de búsqueda de un caso vecino**. El endpoint no re-consulta un término recién servido (es la conducta que TM-655 verifica a propósito): un término compartido dentro de la misma batería hace que tu caso salga `SIN_DATOS` con el producto sano. Nos pasó con TM-654 vs TM-651 (`corrientes`).

B.6 · Dónde vive cada cosa

Carpeta (<code>tests/mobile/appium/</code>)	Rol
<code>config/</code>	Resuelve <code>ENV</code> → <code>env-file</code> , paquete, <code>udid</code> , URL de Appium. El único lugar que decide contra qué ambiente se corre.
<code>base/</code>	<code>AppiumSessionBase</code> : sesión, contextos, búsqueda de elementos, inputs web.
<code>helpers/</code>	<code>webViewNetworkCapture</code> — la pieza que hace medibles los requests.
<code>driver/</code> · <code>passenger/</code>	Pantallas y baterías de casos por actor; <code>specs/</code> adentro son los tests que corre el runner.
<code>harness/</code>	Flujos completos reutilizables (happy path de un viaje punta a punta).
<code>scripts/</code>	Instrumentos de investigación. No son tests : no producen reporte ni cuentan como cobertura.

B.7 · Pendientes conocidos, para quien retome

Qué	Detalle
TM-727 en dispositivo PAX	Los campos Origen/Destino del home son <code>readOnly</code> (abren un buscador aparte). Falta replicar el patrón «alcanzar el buscador» del lado driver. El defecto de orden ya está confirmado por duplicado, así que es cierre de matriz, no descubrimiento.
TM-663 (driver)	Exige seleccionar una predicción; el runner aún no pasa el gancho de selección (<code>onSelect</code>) a la batería.
TM-654 (driver)	El guard de premisa detectó intervalo real > ventana de debounce en la última corrida — repetir en un teléfono menos cargado o bajar <code>keyGapMs</code> .
Caso «orden» del driver	Existe como guard sin key de Xray (decisión deliberada hasta que MG-931 cierre). El reporter lo lista como <i>unmapped</i> : es esperado, no un defecto de la corrida.

Fuentes: artefactos `evidence/uat/xray-results-*-2026-08-24*.json` y corridas citadas por timestamp en el cuerpo.
 Documento generado desde `docs/mobile/GUIA-APPIUM-MG116-MG117.html` — editá el HTML y regenerá el PDF con `node scripts/render-guia-appium-pdf.mjs`.